

深圳大学实验报告

课程名称： 计算机系统(2)

实验项目名称： 数据表示实验

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 马晨琳

报告人： 黄嘉聪 学号： 2023192044 班级： 1

实验时间： 2025年4月2日至4月15日

实验报告提交时间： 2025年4月15日

教务处制

一、实验目的：

1. 了解各种数据类型在计算机中的表示方法
2. 掌握 C 语言数据类型的位级表示及操作

二、实验内容：

1、安装 gcc-multilib:

```
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~$ sudo apt-get install gcc-multilib
[sudo] huangjiacong_2023192044 的密码:
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
将会同时安装下列软件：
gcc-13-multilib lib32asan8 lib32atomic1 lib32gcc-13-dev lib32gcc-s1 lib32gomp1 lib32itm1 lib32libasan8
lib32libatomic1 lib32gcc-13-dev lib32gcc-s1 lib32gomp1 lib32itm1 lib32quadmath0 lib32stdc++13
```

或者：

```
test@szu-VirtualBox:/media/sf_计系2/datalab-handout$ su
Password:
root@szu-VirtualBox:/media/sf_计系2/datalab-handout# apt-get install gcc-multilib
Reading package lists... Done
Building dependency tree
Reading state information... Done
gcc-multilib is already the newest version (4:7.2.0-1ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 84 not upgraded.
```

2、根据 bits.c 中的要求补全以下的函数：

```
int bitXor(int x, int y);
int min(void);
int isTmax(int x);
int allOddBits(int x);
int negate(int x);
int isAsciiDigit(int x);
int conditional(int x, int y, int z);
int isLessOrEqual(int x, int y);
int logicalNeg(int x);
int howManyBits(int x);
unsigned floatScale2(unsigned uf);
int floatFloat2Int(unsigned uf);
unsigned floatPower2(int x);
```

3、在 Linux 下测试以上函数是否正确，指令如下（详见 README 文件）：

*编译：./dlc bits.c

*测试：make btest

./btest

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

三、实验思路及求解过程：

1、安装 gcc-multilib:

如图 1 所示，输入指令【`sudo apt-get install gcc-multilib`】对 gcc-multilib 进行安装

```
huangjiacong_2023192044@chino-VMware-Virtual-Platform:~$ sudo apt-get install gcc-multilib
[sudo] huangjiacong_2023192044 的密码:
正在读取软件包列表... 完成
正在分析软件包的依赖关系树... 完成
正在读取状态信息... 完成
将会同时安装下列软件:
gcc-13-multilib lib32asan8 lib32atomic1 lib32gcc-13-dev lib32gcc-s1 lib32gomp1 lib32itm1 lib32libasan8
lib32libatomic1 lib32libgcc-13-dev lib32libgcc-s1 lib32libgomp1 lib32libitm1 lib32libquadmath0 lib32stdc++-13-dev
```

图 1 安装 gcc-multilib

2、根据 bits.c 中的要求补全以下的函数：

(1) `int bitXor(int x, int y);`

①题目描述：

仅允许使用~和&来实现异或

允许的操作符: ~ &

最多操作符数目: 14

②思路：首先通过异或定义，有【 $x \oplus y = (x \& y) | (\sim x \& y)$ 】，由于不能使用|运算符，应用德摩根律转换为【 $\sim(\sim(\sim x \& y) \& \sim(x \& \sim y))$ 】返回。

③编程实现：

```
125     int bitXor(int x,int y){
126         return ~((~(x & y) & ~(x & ~y)));
127     }
```

图 2 函数 int bitXor(int x, int y)

(2) `int tmin(void);`

①题目描述：返回最小的二进制补码

允许的操作符: ! ~ & ^ | + << >>

最多操作符数目: 4

②思路：32 位最小值二进制为 100...000（其中第 31 位为 1），故通过最低位为 1 的数字 1 左移 31 位即【`1 << 31`】直接构造返回。

③编程实现：

```
136     int tmin(void)
137     {
138         return 1 << 31;
139     }
```

图 3 函数 int tmin(void)

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

(3) int isTmax(int x);

①题目描述：如果 x 是最大的二进制补码，返回 1；否则，返回 0

允许的操作符: $! \sim \& ^ | +$

最多操作符数目: 10

②思路：对于最大补码 $0x7FFFFFFF$ 有 $[x + 1 = \sim x]$ ，故使用 $[!(x + 1 \wedge \sim x)]$ 判断，但在判断之外还需要排除 $0xFFFFFFFF$ 的特殊情况，使用 $[!!(x + 1)]$ 进行判断，故有返回式：

$[!(x + 1 \wedge \sim x) \& !(x + 1)]$

③编程实现：

```
147 int isTmax(int x)
148 {
149     return !((x + 1) ^ (~x)) & !(x + 1);
150 }
```

图 4 函数 int isTmax(int x)

(4) int allOddBits(int x);

①题目描述：如果所有奇数位都为 1 则返回 1;否则返回 0

允许的操作符: $! \sim \& ^ | + << >>$

最多操作符数目: 12

②思路：需要构建 $0xAAAAAAAA$ 掩码，通过 $0xAA$ 逐级右移拼接得到完整掩码，然后根据 $0xAAAAAAAA$ 的性质检查 $x \& mask$ 是否等于 $mask$ 。

③编程实现：

```
160 int allOddBits(int x)
161 {
162     int mask = 0xAA;
163     mask = mask | (mask << 8);
164     mask = mask | (mask << 16);
165     return !((x & mask) ^ mask);
166 }
```

图 5 函数 int allOddBits(int x)

(5) int negate(int x);

①题目描述：返回 $-x$

允许的操作符: $! \sim \& ^ | + << >>$

最多操作符数目: 5

②思路：有补码相反数公式 $[\sim x + 1]$ ，直接返回即可。

③编程实现：

```
177 int negate(int x)
178 {
179     return ~x + 1;
180 }
```

图 6 函数 int negate(int x)

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

(6)int isAsciiDigit(int x);

①题目描述: * isAsciiDigit - 如果 x 是 ascii 码中的 0~9, 返回 1;否则返回 0

允许的操作符: ! ~ & ^ | + << >>

最多操作符数目: 15

②思路: 检查x是否在0x30 - 0x39的范围内, 通过判断其正负即计算 $x - 0x30$ 和 $0x39 - x$ 的符号位进行判断返回。

③编程实现:

```
192 int isAsciiDigit(int x)
193 {
194     int lowerBound = x + (~0x30 + 1); //x-0x30
195     int upperBound = 0x39 + (~x + 1); //x-0x39
196     return !(lowerBound >> 31) & !(upperBound >> 31);
197 }
```

图 7 函数 int isAsciiDigit(int x)

(7)int conditional(int x, int y, int z);

①题目描述: 实现 $x ? y : z$

允许的操作符: ! ~ & ^ | + << >>

最多操作符数目: 16

②思路: 先构造掩码有 $(!x) + (\sim 0)$, 其非零时全 1, 否则全 0, 然后使用算术右移特性: $mask = (!x + 0)$, 最后类似的返回 $(mask \& y) | (\sim mask \& z)$ 得到 $x ? y : z$ 。

③编程实现:

```
207 int conditional(int x, int y, int z)
208 {
209     int mask = (!x) + (~0);
210     return (mask & y) | (~mask & z);
}
```

图 8 函数 int conditional(int x, int y, int z)

(8)int isLessOrEqual(int x, int y);

①题目描述: 如果 $x \leq y$ 返回 1 否则返回 0

允许的操作符: ! ~ & ^ | + << >>

最多操作符数目: 24

②思路: 分三种情况处理: 在不同符号时直接判断; 如果同符号时计算 $x - y$ 的符号; 最后处理相等的情况。公式如图 9 所示。

③编程实现:

```
221 int isLessOrEqual(int x, int y)
222 {
223     int signX = x >> 31;
224     int signY = y >> 31;
225     return (signX & !signY) | ((!(signX ^ signY)) & ((x + ~y) >> 31 & 1)) | !(x ^ y);
226 }
```

图 9 函数 int isLessOrEqual(int x, int y)

注: 1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

(9)int logicalNeg(int x);

①题目描述：* logicalNeg - 实现！运算符的功能

允许的操作符：~ & ^ | + << >>

最多操作符数目：12

②思路：利用 x 和 $-x$ 的符号位特征：即当且仅当 $x = 0$ 时， $x | -x$ 的符号位为0。此时通过右移31位后取反即可得到结果并返回。

③编程实现：

```
238     int logicalNeg(int x)
239     {
240         return ((x | (~x + 1)) >> 31) + 1;
241     }
```

图 10 函数 int logicalNeg(int x)

(10)int howManyBits(int x);

①题目描述：返回将 X 表示为补码所需的最小有效位数。

允许的操作符：! ~ & ^ | + << >>

最多操作符数目：90

②思路：通过二分法查找 x 的最高有效位，即通过负数先取反、多次移位和mask操作定位最高位、最后特殊处理0和-1的情况、再累加各阶段移位量后返回。

③编程实现：

```
255     int howManyBits(int x) {
256         int shift1, shift2, shift4, shift8, shift16;
257         int sum;
258         int t = ((!x) << 31) >> 31;    //如果x为0，则t全为1，否则全为0
259         int t2 = ((!~x) << 31) >> 31;  //如果x为-1，则t2全为1，否则全为0
260         int op = x ^ ((x >> 31));      //如果x为负，则取反；如果x为正，保持不变
261         shift16 = (!!(op >> 16)) << 4;
262         op = op >> shift16;
263         shift8 = (!!(op >> 8)) << 3;
264         op = op >> shift8;
265         shift4 = (!!(op >> 4)) << 2;
266         op = op >> shift4;
267         shift2 = (!!(op >> 2)) << 1;
268         op = op >> shift2;
269         shift1 = (!!(op >> 1));
270         op = op >> shift1;
271         sum = 2 + shift16 + shift8 + shift4 + shift2 + shift1;
272         //对于特殊情况：0返回1，-1返回1，最小负数返回32
273         return (t2 & 1) | ((~t2) & ((t & 1) | ((~t) & sum)));
274     }
```

图 11 函数 int howManyBits(int x)

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后10日内。

(11) unsigned floatScale2(unsigned uf);

①题目描述：以 unsigned 表示的浮点数二进制的二倍的二进制 unsigned 型参数和结果都会被作为 unsigned 返回，但是会表示为二进制的单精度浮点值。

允许的操作符：任何整数或者无符号数操作符包括：||, &&, also if, while

最多操作符数目：30

②思路：分三种情况处理：遇到输入位NaN和无穷大直接返回；对于非规格化数：尾数左移；对于规格化数：指数+1；最后处理尾数溢出到指数位的特殊情况。

③编程实现：

```
284 unsigned floatScale2(unsigned uf)
285 {
286     unsigned sign = uf & 0x80000000;
287     unsigned exp = (uf & 0x7F800000) >> 23;
288     unsigned frac = uf & 0x007FFFFFFF;
289     if (exp == 0xFF)
290     {
291         return uf;
292     }
293     if (exp == 0)
294     {
295         frac <<= 1;
296         return sign | frac;
297     }
298     exp++;
299     if (exp == 0xFF)
300     {
301         return sign | 0x7F800000;
302     }
303     return sign | (exp << 23) | frac;
304 }
```

图 12 函数 unsigned floatScale2(unsigned uf)

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

(12) `int floatFloat2Int(unsigned uf);`

①题目描述：返回 `unsigned uf` 的整型数的二进制形式

参数和结果都会被作为 `unsigned` 返回，但是会表示为二进制的单精度浮点值

任何超过范围的数都应该返回 `0x80000000u`。

允许的操作符：任何整数或者无符号数，操作符包括： `||`, `&&`, `also if`, `while`

最多操作符数目：30

②思路：对于输入的数组，分离其符号位、指数位、尾数位、并处理特殊值的情况（即 *NaN*、无穷大）、然后计算实际指数 E 。

再根据 E 范围处理： $E < 0$ 返回 0、 $E > 31$ 返回溢出、其余的 $0 < E < 31$ 重构尾数并调整位数

③编程实现：

```
315  int floatFloat2Int(unsigned uf)
316  {
317      unsigned sign = uf & 0x80000000;
318      unsigned exp = (uf & 0x7F800000) >> 23;
319      unsigned frac = uf & 0x007FFFFF;
320      if (exp == 0xFF)
321      {
322          return 0x80000000u;
323      }
324      frac |= 0x00800000;
325      if (e > 23)
326      {
327          frac <<= (e - 23);
328      }
329      else
330      {
331          frac >>= (23 - e);
332      }
333      if (sign)
334      {
335          return ~frac + 1;
336      }
337      else
338      {
339          return frac;
340      }
341  }
```

图 13 函数 `int floatFloat2Int(unsigned uf)`

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

(13) unsigned floatPower2(int x);

①题目描述：返回 2 的 x 次方的浮点数值

(无符号)二进制表示形式

如果结果太小而无法表示，返回 0

如果结果太大而无法表示，返回 +inf

允许的操作符：任何整数/无符号运算 包括 ||, &&. also if, while

最多操作符数目: 30

②思路：分情况处理： $x < -149$ ：返回 0； $-149 \leq x < -126$ ：非规格化数； $-126 \leq x \leq 127$ ：规格化数； $x > 127$ ：返回无穷大、最后根据指数范围构造结果。

③编程实现：

```
363 unsigned floatPower2(int x) {
364     unsigned result;
365     if (x < -126) {
366         return 0;
367     }
368     if (x > 127) {
369         result = 0x7F800000;
370         return result;
371     }
372
373     if (x >= 0) {
374         result = (x + 127) << 23;
375     } else {
376         result = (x + 126) << 23;
377     }
378     return result;
379 }
```

图 14 unsigned floatPower2(int x)

3、在 Linux 下测试以上函数是否正确，指令如下（详见 README 文件）：

*编译：make

*测试：./btest

(1)在代码的目录下输入命令【make】，如图 15 所示。代码编译成功。

```
huangjiacong_2023192044@chino-VMware-Virtual-Platform:/mnt/hgfs/datalab-handout$ make
gcc -O -Wall -m32 -lm -o btest bits.c btest.c decl.c tests.c
btest.c: In function 'test_function':
btest.c:334:23: warning: 'arg_test_range' may be used uninitialized [-Wmaybe-uninitialized]
 334 |     if (arg_test_range[2] < 1)
      |         ^~~~~~
btest.c:299:9: note: 'arg_test_range' declared here
 299 |     int arg_test_range[3]; /* test range for each argument */
      |         ^~~~~~
./huangjiacong_2023192044@chino-VMware-Virtual-Platform:/mnt/hgfs/datalab-handout$ ./btest
```

图 15 使用 make 指令进行编译

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

(2), 输入指令【./btest】运行函数测试。如图 16 所示, 函数测试成功并满分, 说明函数无问题, 实验完成。

```
huangjiacong_2023192044@chino-VMware-Virtual-Platform:/mnt/hgfs/dataLab-handout$ ./btest
Score  Rating  Errors  Function
1      1       0      bitXor
1      1       0      tmin
1      1       0      isTmax
2      2       0      allOddBits
2      2       0      negate
3      3       0      isAsciiDigit
3      3       0      conditional
3      3       0      isLessOrEqual
4      4       0      logicalNeg
4      4       0      howManyBits
4      4       0      floatScale2
4      4       0      floatFloat2Int
4      4       0      floatPower2
Total points: 36/36
```

图 16 使用./btest 指令进行函数测试

四、实验结论及问题：

- ①通过本次数据表示实验, 深入理解了计算机中整数与浮点数的二进制表示原理及位级操作方法。
- ②在实验当中, 通过补全 13 个 C 语言函数, 系统实践了位运算、补码特性及 IEEE 754 标准的核心机制。例如, 利用德摩根律实现异或运算 (bitXor), 通过掩码构建与位移操作验证奇数位全 1 (allOddBits), 结合符号位判断与算术右移完成条件选择 (conditional) 等, 强化了对位级操作的灵活运用能力。
- ③在浮点数处理部分, 通过分离符号位、指数位与尾数位, 掌握了非规格化数与规格化数的转换逻辑 (如 floatScale2、floatPower2), 并实现了浮点数与整型数的双向转换 (floatFloat2Int), 进一步理解了浮点运算的精度限制与溢出处理。实验过程中, 通过多次调试与边界条件测试 (如处理 Tmax、0x80000000 等特殊值), 加深了对数据表示边界情况的认知。
- ④最终通过提供的自动化测试验证了所有函数的正确性, 验证了理论分析的可行性。
- ⑥需要说明的是, 对于最后一个函数, 在使用测试进行验证时出现了明明无任何循环语句却运行超时的情况, 最后通过对测试代码进行翻阅、验证等, 发现为在 del.c 文件当中, 其配置有超出应有测试数据的情况, 最后通过修改其至正常值, 使得测试功能正常运行, 完成实验。
- ⑥本次实验不仅巩固了计算机系统中数据表示的底层原理, 还提升了通过位操作、二进制操作解决实际问题的能力, 为后续深入理解计算机系统架构奠定了实践基础。

注：1、报告内的项目或内容设置, 可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。

指导教师批阅意见：
成绩评定：
指导教师签字：
2023 年 4 月 日
备注：

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。